

国产众核高性能计算机系统 神威应用辅助开发库使用手册

国家并行计算机工程技术研究中心

二零二零年十二月九日

目录

第一章、离散访存库	5
一、简介.....	5
二、使用方法.....	5
三、编程接口.....	5
1、主核接口	5
2、从核接口	9
四、程序示例.....	12
五、完整程序示例.....	13
六、离散访存库性能分析.....	16
第二章、SWAF_PCG 求解器	18
一、使用方法.....	18
二、接口说明.....	18
第三章、稀疏矩阵向量乘.....	24
一、使用方法.....	24
二、数据类型.....	24
三、编程接口.....	24
1、主核接口	24
2、从核接口	25
四、程序示例.....	29
1、COO 格式矩阵向量乘法	29
2、CSR 格式矩阵向量乘法	32
五、性能测试.....	35
第四章、稠密矩阵乘使用手册.....	36
一、简介.....	36
二、使用方法.....	36
三、编程接口.....	36
四、性能数据.....	43
1、单精度	43
2、半精度	43
第五章、张量转置	45
一、简介.....	45
二、使用方法.....	45
三、编程接口.....	45
四、程序示例.....	46
五、性能对比.....	49
第六章、窄条矩阵乘	50
一、使用方法.....	50

二、函数接口.....	50
三、程序示例.....	50
四、性能对比.....	52
第七章、三维矩阵转置.....	53
一、使用方法.....	53
二、函数接口.....	53
三、程序示例.....	53
四、性能对比.....	55
第八章、稀疏对角矩阵方程组求解	56
一、使用方法.....	56
二、函数接口.....	56
三、程序示例.....	57
四、性能对比.....	59
附录 1：离散访存库应用实例	60
一、非规则访存问题.....	60
二、接口使用.....	60
三、性能模型与分析.....	61
四、性能测试.....	62
附录 2：技术支持	63

第一章、离散访存库

一、简介

离散访存库提供一种便捷的非规则数据众核优化方法。主要针对一个从核 LDM 放不下的离散数据，通过简单的几个接口函数，实现任一从核快速访问其他从核 LDM 空间的功能。离散访存库特有的信息关系预存技术，特别适合离散访问位置不变，访问数据变化的应用，如非结构网格类应用。在该类应用中，网格拓扑结构决定了离散访问位置，对于某算例，网格不变离散访问位置自然固定不变，而离散访问的数据随着迭代进行不断变化，这种情况非常适用本文给出的离散访存库。

二、使用方法

SWAF_DCL 使用 C 语言开发，目前暂提供 C 语言接口。使用时需要包含/usr/lib/SWAF 路径下 include 目录下的 SWAF_dcl.h,并链接 lib 目录下的 libSWAF_dcl.a。

三、编程接口

1、主核接口

1) **void SWAF_prePosmod(int *d, int N, int M, int allsize, struct DIS_TYPEDATA *pos2global);**

函数说明：

对单层循环计算问题的离散访存下标预处理。对于计算过程中离散下标保持不变的数组，在进行迭代计算之前进行一次下标预处理，然后在计算过程中可反复使用预处理模型。

参数说明：

输入：

d: 离散访存索引表;

N: 循环计算量，即计算过程中离散访存次数;

M: 离散访存数据总量，即计算过程中离散访存的数据的数量或范围。例如 $F[X]$ 是要离散访问的数组，那么 M 为 X ；如果数组是 $F[X][Y]$ ，其中对第一维离散访问，对第二维是连续的访问，那么 M 为 X ；

allsize: 与离散数据相关元素的总数。例如 $F[X]$ 是要离散访问的数组，那么 **allsize** 为 X ；如果数组是 $F[X][Y]$ ，其中对第一维是离散访问的，对第二维是连续的访问，那么 **allsize** 为 $X*Y$ ；

输出:

pos2global: 将预处理后建立的相关映射表保存至该结构体中。

Exp1:

<pre>1. struct DIS_TYPEDATA str_pos 2. int d[N] 3. double A[M], B[N], C[M][V] 4. for i 0 -> N 5. for j 0 -> V 6. A[i] += B[i] * C[d[i]][j]; 7. endfor 8. Endfor</pre>

说明：对 C 离散的访问进行预处理。

C 的数据是离散的，需对 d 进行预处理：

<pre>1. SWAF_prePosmod(&d[], N, M, M*V, &str_pos)</pre>

2) **void SWAF_prePosmod2D(int *r, int *d, int N, int M, int allsize, struct DIS_TYPEDATA *pos2global);**

函数说明:

对两层循环计算问题的离散访存索引预处理。对于计算过程中离散索引保持不变的数组，在进行迭代计算之前进行一次预处理，在计算过程中可反复使用预处理模型。

参数说明:

输入:

r: 低维计算索引表；

d: 离散访存索引表;

N: 循环计算量, 即计算过程中离散访存的次数;

M: 离散访存数据总量;

allsize: 与离散数据相关元素的总数;

输出:

pos2global: 将预处理后所建立相关映射表保存至该结构体中。

Exp2:

<pre>1. struct DIS_TYPEDATA str_pos 2. int d[N], r[N+1] 3. double A[M], B[N], C[M][V] 4. for i 0 -> N 5. for j r[i] -> r[i+1] 6. A[d[i]] += B[i] * C[d[i]][V] 7. endfor 8. endfor</pre>
对离散数据索引 d 进行预处理: SWAF_prePosmod2D(r, d, N, N, V, &str_pos)

3) **void SWAF_prePosmodmal(int *d, int N, int M, int grid_block, int cal_talsize, struct DIS_TYPEDATA *pos2global);**

函数说明:

对单层循环内计算问题的手动可调节式离散访存下标预处理。对于需要分块的从核并行计算, 可手动设置从核分块信息, 包括对离散访存总量分块与计算量的分块。

参数说明:

输入:

d: 离散访存表;

N: 循环计算量，即计算过程中离散访存的次数；

M: 离散访存数据总量；

grid_block: 离散访存数据块；从核阵列一次存放离散访存数据量大小，当 LDM 空间一次无法存放所有离散访存数据量时，可对访存量进行分块存放；【默认值：163840】；

cal_talsize: 计算分块；从核阵列一次处理的计算量大小，当 LDM 空间一次无法处理所有计算时，可对计算量进行分块；【默认值为：524288】；

输出：

pos2global: 将预处理后所建立相关映射表保存至该结构体中。

Exp3:

1. struct DIS_TYPEDATA str_pos
2. int d[N]
3. double A[M], B[N], C[M]
4. for i 0 -> N
5. A[d[i]] += B[i] * C[d[i]]
6. endfor
对离散数据索引 d 进行预处理：
1. SWAF_prePosmodmal(d, N, M, M, 81920, &str_pos)

4) **void SWAF_prePosmod2Dmal(int *r, int *d, int N, int M, int grid_block, int cal_talsize, struct DIS_TYPEDATA *pos2global);**

函数说明：

对二层嵌套循环计算问题的离散访存下标预处理。对于需要分块的从核并行计算，可手动设置从核分块信息，包括对离散访存总量分块与计算量的分块。

参数说明：

输入：

r: 低维计算下标偏移表；

d: 离散访存表;

N: 循环计算量;

M: 离散访存数据总量;

grid_block: 离散访存数据块;

cal_talsize: 计算分块;

输出:

pos2global: 将预处理后所建立相关映射表保存至该结构体中。

Exp4:

<pre>1. struct DIS_TYPERDATA str_pos 2. int d[N], r[N+1] 3. double A[M], B[N], C[M][V] 4. for i 0 -> N 5. for j r[i] -> r[i+1] 6. A[d[i]] += B[i] * C[d[i]][V] 7. endfor 8. endfor</pre>
对离散数据索引 d 进行预处理:
<pre>1. SWAF_prePosmod2Dmal(r, d, N, N, 262144, 81920, &str_pos)</pre>

5) **void SWAF_pre_join();**

函数说明:

预处理阻塞等待接口，等待从核预处理结束。

2、从核接口

1) **void SWAF_setArea(struct DIS_TYPERDATA * pos2global, int *slave_loop_size);**

函数说明:

设置从核并行分块：从核计算前调用该接口获取已划分的本地从核计算分块；即当从核处理所有计算时，对计算量进行分块；

参数说明：

输入：

pos2global：坐标映射模型，存放离散数据对应从核阵列映射信息；

输出：

slave_loop_size：本地从核数据并行分块数。

2) **void SWAF_getLocInfo(int i_block, int *blocksize, int *master_start, int *master_end);**

函数说明：

获取从核当前分块下的计算量，和当前分块下对应原始主核循环段索引起始值。

参数说明：

输入：

i_block：并行分块下当前分块编号；

输出：

blocksize：当前分块下计算量；

master_start：当前分块对应原始主核循环段下表起始值；

master_end：当前分块对应原始主核循环段下表终止位置。

3) **void SWAF_readD(double *C, double *s_C, int i_block, int blocksize, int vec_size, struct DIS_TYPEDATA *pos2global);**

函数说明：

将计算所需的原始离散数据块保存至从核 LDM 缓存空间中（对应：result_slave_data 变量），使原始离散数据变为本地连续数据块；

参数说明：

输入：

C: 主存上的原始离散数据；

i_block: 并行分块下当前分块编号；

blocksize: 当前分块下离散数据计算量；

vec_size: 对应向量块长度；

pos2global: 坐标映射模型；

输出：

s_C: 原离散数据连续化后存储在从核 LDM 的位置。

4) **void SWAF_wroteD(double *s_A, int op, int i_block_i, int blocksize, int vec_size, struct DIS_TYPEDATA *pos2global, double *A);**

函数说明：

将本地计算后的数据批量更新至主存原始数据；

参数说明：

输入：

s_A: 需要更新的本地离散数据块；

op: 操作符。【当前暂支持加法运算，传值为 DIS_PLU】

i_block: 并行分块下当前分块编号;

blcoksize: 当前分块下离散数据计算量;

vec_size: 对应向量块长度;

pos2global: 坐标映射模型;

输出:

A: 需要更新的主存目标数据。

四、程序示例

针对一层循环和二层循环两类计算问题，分别给出以下主从核关键步骤:

一层循环:

- 1. for I 0 -> N
- 2. A[d[i]] += B[i] * C[d[i]]
- 3. endfor

描述: 对 A,C 的访问是离散的，需要对 A,C 访问预处理，在计算时由离散访问转化为本地连续访问;

主从接口使用:

主核	从核
1. DIS_TYPEDATA buf; 2. SWAF_prePosmod(&d[0], N, M, DIS_REAL8, &buf) 3. SWAF_pre_join();	1. SWAF_setArea(&buf, &block_n); 2. For I 0 -> block_n 3. SWAF_getLocInfo(i, &bsize, &mst, &med); 4. s_C = alloc; 5. SWAF_read(&C[0], I,bsize , 1, & buf, &s_C [0]); 6. for j 0 -> bsize 7. s_A[j] = s_B[j] * s_C[j]; 8. endfor 9. SWAF_write(&s_A[0], I,bsize , 1, & buf, &A[0]); 10. endfor

二层循环:

- 1. for I 0 -> N

```
2. for J r[l] -> r[l+1]
3.     A[i] += B[j] * C[d[j]]
4. endfor
5. endfor
```

描述：对 C 访问离散，对 C 访问预处理，在计算时由离散访问转化为本地连续访问;

主从接口使用：

主核	从核
1. DIS_TYPEDATA buf; 2. SWAF_prePosmod2D(&r[0], &d[0], N, M, DIS_REAL8, &buf) 3. SWAF_pre_join();	1. SWAF_setArea(&buf, &block_n); 2. For l 0 -> block_n 3. SWAF_getLocInfo(i, &bsize, &mst, &med); 4. s_C=malloc; 5. SWAF_read(&C[0], l, bsize, 1, &buf, &s_C [0]); 6. for j mst -> med 7. for k s_r[mst] -> s_r[mst+1] 8. s_A[j] += s_B[j] * s_C[icnt++]; 9. endfor 10. endfor 11. endfor

五、完整程序示例

使用离散访存库分为主核与从核两部分，主核程序需引用“dis_master.h”，从核程序需引用“dis_slave.h”。

1、原始纯主核程序：

```
1. #include "stdio.h"
2. #include "stdlib.h"
3.
4. #define MAXT 1e0
5. uint64_t x=1,y=7;
6.
7. uint64_t my_rand(){
8.     x ^= x<<29;
9.     x ^= x>>23;
10.    x ^= y ^ (y >> 31);
11.    return x+y;
12. }
13. double A[N];
14.
```

```
15. void main(){
16.     int idx[M];
17.
18.     for(i=0; i<M; i++){
19.         int tp = my_rand() % N;
20.         idx[i] = tp;
21.     }
22.
23.     for(i = 0; i < N; i++){
24.         int vt = my_rand() % N;
25.         A[i] = 3.14*i + 0.27 * vt;
26.     }
27.
28.     double *result = (double *)malloc(M *sizeof(double));
29.     memset(result, 0, M *sizeof(double));
30.
31.     cyclet[0] = CRTS_time_cycle();
32.     for(i = 0; i < M; i++){
33.         result[i] += A[idx[i]];
34.     }
35. }
```

2、众核程序：

1) 主核

```
1.  #include "stdio.h"
2.  #include "stdlib.h"
3.  #include "crt.h"
4.  #include "dis_master.h"
5.
6.  #define MAXT 1e0
7.  uint64_t x=1,y=7;
8.
9.  uint64_t my_rand(){
10.     x ^= x<<29;
11.     x ^= x>>23;
12.     x ^= y ^ (y >> 31);
13.     return x+y;
14. }
15. double A[N];
16. double *result1;
17. int *row;
```

```

18. struct DIS_TYPEDATA m_buff;
19.
20. extern SLAVE_FUN(func)(double *);
21. void main(){
22.     int idx[M];
23.     SWAF_init();
24.
25.     for(i=0; i<M; i++){
26.         int tp = my_rand() % N;
27.         idx[i] = tp;
28.     }
29.
30.     for(i = 0; i < N; i++){
31.         int vt = my_rand() % N;
32.         A[i] = 3.14*i + 0.27 * vt;
33.     }
34.
35.     result1 = (double *)malloc(M *sizeof(double));
36.     memset(result1, 0, M *sizeof(double));
37.     SWAF_prePosmod(&idx[0], M, N, DIS_REAL8, &m_buff);
38.     SWAF_pre_join();
39. }

```

2) 从核

```

1.  #include "stdio.h"
2.  #include "stdlib.h"
3.  #include <string.h>
4.  #include "crts.h"
5.
6.  #include "dis_slave.h"
7.
8.  extern double A[N];
9.  extern double *result1;
10. extern struct DIS_TYPEDATA m_buff;
11.
12. __thread_local volatile int dma_reply=0;
13. __thread_local double *sum, *vp_s;
14.
15. void func(){
16.
17.     int block_st, i_block, pos_size;
18.     int block_n;
19.

```

```
20.   SWAF_setArea(&m_buff, &block_n);
21.   for(i_block = 0; i_block<block_n; i_block++){
22.
23.       SWAF_getLocInfo(i_block, &pos_size, &block_st, &block_end);
24.
25.       double *sbuf = (double *)ldm_malloc((2*pos_size+4)*sizeof(double));
26.       sum = &sbuf[0];
27.       vp_s = &sbuf[pos_size+1];
28.       memset(sum, 0, pos_size*sizeof(double));
29.
30.       SWAF_readD(&A[0], i_block, pos_size, 1, &m_buff, &vp_s[0]);
31.
32.       for(int ii= 0 ; ii < pos_size; ii++){
33.           sum[ii] += vp_s[ii];
34.       }
35.
36.       CRTS_dma_put(&result1[block_st], &sum[0], pos_size*sizeof(double));
37.       ldm_free(&sbuf[0],(2*pos_size+4)*sizeof(double));
38.   }
39. }
40. }
```

六、离散访存库性能分析

1、离散读

```
#define V 4
double A[N][V]

1.  for(i = 0; i < M; i++){
2.     double tmp = 0;
3.     for( j = 0; j < V; j++)
4.         tmp += B[i] * A[idx[i]][j];
5.     result[i] = tmp;
6. }
```

针对上述计算代码，不同规模下使用离散访存库与共享 LDM 模式，单核组加速比如下：

算例编号	N	M	采用离散访存库单核组加速比	采用共享 ldm 单核组加速比
1	20000	200000	28	17
2	40000	400000	35	23
3	80000	800000	35	27
4	160000	1600000	33	30

5	320000	3200000	24	无
6	640000	6400000	23	无

2、离散写

```
double result[N]

1.  for(i = 0; i < M; i++){
2.      result[idx[i]] += B[i] * A[i];
3.  }
```

针对上述代码，不同规模下使用离散访存库单核组加速比如下：

算例编号	N	M	采用离散访存库单核组加速比
1	20000	200000	15
2	40000	400000	16
3	80000	800000	19
4	160000	1600000	22
5	320000	3200000	21
6	640000	6400000	21

6. 离散访存库与共享 LDM 模式对比总结

1. 离散访存库不需要指定固定大小共享 LDM 空间，可节省 LDM 空间；
2. 共享 LDM 目前只支持静态数据类型定义，离散访存库不受限制；
3. 离散访存库不需要在作业管理系统下设置共享参数；
4. 离散访存库不受离散访存规模限制，共享 LDM 目前支持单核最大 128KB 共享空间；
5. 共享 LDM 模式需要用户手动解决离散写冲突问题，离散访存库已为用户封装好可避免写冲突问题的离散访存写数据接口；
6. 共享 LDM 模式中的共享数据 DMA PUT 暂不支持任意数据量，离散访存库 DMA 写功能不受规模限制；
7. 对于一般性离散访存计算问题，离散访存库已为用户设置好并行编程框架，只需要调用相应参数即可进行计算，并且普遍具有较高性能。

第二章、SWAF_PCG 求解器

一、使用方法

SWAF_PCG 使用 C 语言开发，目前暂提供 C 语言接口，Fortran 和 c++接口将在后续版本中如需提供。

使用时需要包含/usr/lib/SWAF 路径下 include 目录下的 SWAF_pcg.h,并链接 lib 目录下的 libSWAF_pcg.a

二、接口说明

SWAF_PCG 解法器支持 coo, csr 两种存储格式的线性方程组求解（即求解 $Ax=b$ 问题）。在对方程组求解前，需要先对矩阵 A 进行预处理。

(1) SWAF_PCG_setMatIndex(const long int *uidx, const long int *lidx, int gridsize, int elemsize);

函数说明：

使用 coo 存储格式矩阵预处理。

参数说明：

uidx: 上三角矩阵对应非零元列号；

lidx: 下三角矩阵对应非零元列号；

gridsize: 对应原始矩阵列数；

elemsize: 上（下）三角矩阵对应非零元个数。

(2) SWAF_PCG_CSRpreprocess(long int *row_offset, long int *col_index, int mat_colsize, int mat_rowsize);

函数说明:

使用 csr 存储格式矩阵预处理。

参数说明:

row_offset: 矩阵行偏移;

col_index: 矩阵列索引;

mat_colsize: 原矩阵列数;

mat_rowsize: 原矩阵行数。

```
(3) SWAF_PCGcoo (const int gridsize, const int elemsize, const long int *uidx, const long int *lidx, const double *lower, const double *upper, const double *diagonal, const double *src_b, int maxIter, double tolerance, double normFactor, double *x );
```

函数说明:

对矩阵 A 预处理之后,可对不同格式的矩阵分别使用不同的求解接口对方程组求值。使用 coo 存储格式方程组求解。

参数说明:

gridsize: 对应原始矩阵列数;

elemsize: 上(下)三角矩阵对应非零元个数;

uidx: 上三角矩阵对应非零元列号;

lidx: 下三角矩阵对应非零元列号;

lower: 下三角矩阵非零元;

upper: 上三角矩阵非零元;

diagonal: 对角矩阵元素;

src_b: 原方程组右端项;

maxIter: 允许最大迭代次数;

tolerance: 收敛残差值;

normFactor: 收敛残差校对因子 (默认值为 1.0)。

返回值:

X: 原方程组得解。

```
(4) SWAF_PCGcsr(const int gridsize, const long int *row_offset, const long int
*col_index, const double *mat_val, const double *src_b, int maxIter, double tolerance,
double normFactor, double *x );
```

函数说明:

使用 csr 存储格式的方程组求解。

参数说明:

gridsize: 对应原始矩阵列数;

row_offset: 矩阵行偏移;

col_index: 矩阵列索引;

mat_val: 原矩阵非零元素;

src_b: 原方程组右端项;

maxIter: 允许最大迭代次数;

tolerance: 收敛残差值;

normFactor: 收敛残差校对因子 (默认值为 1.0)。

返回值:

X: 原方程组得解。

(5) SWAF__coo2csr(double *upper, double *lower, double *diagonal, const long int *uidx, const long int *lidx, const long int elemsize, const long int gridsize);

为提高用户好用性。SWAF_PCG 提供矩阵 coo 格式转化为 csr 格式的相应接口。函数说明:

将 coo 格式存储的矩阵转化为 csr 格式存储。

参数说明:

upper: 上三角矩阵非零元;

lower: 下三角矩阵非零元;

diagonal: 对角矩阵非零元;

uidx: 上三角矩阵对应非零元列号;

lidx: 下三角矩阵对应非零元列号;

elemsize: 上(下)三角矩阵对应非零元个数;

gridsize: 对应原始矩阵列数;

(6) SWAF__coo2csr_onlymatv(double * upper, double *lower, double *diagonal, const long int *uidx, const long int *lidx, int elemsize, int gridsize);

函数说明:

在实际应用过程中，通常矩阵的非零元坐标不会发生变化，而值会发生改变，在首次使用格式转化接口（5）后，可使用此接口仅对矩阵非零元数值进行格式重载，简化格式转化过程。仅对 coo 格式存储的矩阵非零元数值进行转化，转化为 csr 格式。

参数说明：

upper: 上三角矩阵非零元；

lower: 下三角矩阵非零元；

diagonal: 对角矩阵非零元；

uidx: 上三角矩阵对应非零元列号；

lidx: 下三角矩阵对应非零元列号；

elemsize: 上（下）三角矩阵对应非零元个数；

gridsize: 对应原始矩阵列数；

上述 5)，6) 的两种格式转化接口均为半隐式，如需可见转化后的数据结构，可使用下述接口获取数据：

(7) `get_csrRowoffset(long int **row)`

函数说明：

获取 csr 格式行偏移。

参数说明：

row: 对应 csr 格式行偏移。

(8) `get_csrColIndex(long int **col);`

函数说明：

获取 csr 格式列索引。

参数说明：

col：对应 csr 格式列索引。

(9) `get_csrVal(double **val);`

函数说明：

获取 csr 格式非零元数据。

参数说明：

val：对应 csr 格式非零元。

第三章、稀疏矩阵向量乘

一、使用方法

SWAF_SPMV 使用 C 语言开发，目前提供 C 语言 double 类型数据接口，计划未来版本中提供 Fortran 以及其他数据类型接口。

使用时需要包含/usr/lib/SWAF 路径下 include 目录下的 SWAF_spmv.h,并链接 lib 目录下的 libSWAF_spmv.a。

二、数据类型

T_data 矩阵非零元数据类型，目前仅支持 double。

T_idx 矩阵非零元数据行索引，列索引数据类型。

三、编程接口

1、主核接口

```
(1) void SWAF_spmv_coo_master_d(int nnz, int *rows, int *cols, double *data, double *x, double *y);
```

函数说明：

COO 存储格式稀疏矩阵向量乘法。

参数说明：

nnz:非零元个数；

rows:稀疏矩阵行索引号指针；

cols:稀疏矩阵列索引号指针；

data:稀疏矩阵非零元元素值指针；

x: 向量指针;

y: 稀疏矩阵向量乘计算结果指针。

```
(2) void SWAF_spmv_csr_master_d(int n_rows, int *rows_off, int *cols, double *data,  
double *x, double *y);
```

函数说明:

CSR 存储格式稀疏矩阵向量乘法。

参数说明:

n_rows: 稀疏矩阵行数;

rows_off: 稀疏矩阵行偏移指针;

cols: 稀疏矩阵列索引号指针;

data: 稀疏矩阵非零元元素值指针;

x: 向量指针;

y: 稀疏矩阵向量乘计算结果指针。

2、从核接口

```
(1) void SWAF_spmv_init_coo_d(int n_rows, int n_cols, int nnz, T_idx *rows, T_idx  
*cols, T_data *data, spmv_data_t *_data);
```

函数说明:

COO 存储格式稀疏矩阵数据预处理。

参数说明:

n_rows:稀疏矩阵行数;

n_cols:稀疏矩阵列数;

nnz:非零元个数;

rows:稀疏矩阵行索引号指针;

cols:稀疏矩阵列索引号指针;

data:稀疏矩阵非零元元素值指针;

_data:分块数据, 用作稀疏矩阵向量乘法输入。

(2) void SWAF_spmv_init_csr_d(int n, double *value, int *colind, int *rbegin, spmv_data_t *_data);

函数说明:

CSR 存储格式稀疏矩阵数据预处理。

参数说明:

n:稀疏矩阵行数;

colid:稀疏矩阵列索引号指针;

value:稀疏矩阵非零元元素值指针;

rbegins:稀疏矩阵行偏移指针;

_data:分块数据, 用作稀疏矩阵向量乘法输入;

(3) void SWAF_spmv_finish(spmv_data_t *_data);

函数说明:

销毁函数：销毁存储空间。

参数说明：

_data:分块数据，用作稀疏矩阵向量乘法输入。

```
(4) int read_MTX(T_data **data, T_idx **rows, T_idx **cols, int *nnz, int *n_rows,
int *n_cols, char *fname);
```

函数说明：

数据文件读取函数：MTX 数据文件。

参数说明：

data:稀疏矩阵非零元元素值指针；

rows:稀疏矩阵行索引号指针；

cols:稀疏矩阵列索引号指针；

nnz:非零元个数指针；

n_rows:稀疏矩阵行数指针；

n_cols:稀疏矩阵列数指针；

fname:读取文件指针。

```
( 5 ) int write_MTX(T_data *data, T_idx *rows, T_idx *cols, int nnz, char
*file_name, char file_name_wr[MAX_BUFFER]);
```

函数说明：

数据文件写函数：MTX 文件。

参数说明:

data:稀疏矩阵非零元元素值指针;

rows:稀疏矩阵行索引号指针;

cols:稀疏矩阵列索引号指针;

nnz:非零元个数;

file_name:读取文件指针;

file_name_wr:写文件全路径。

(6) void SWAF_spmv_o2r_d(int nnz, int n_rows, T_idx *rows, T_idx **rows_off);

函数说明:

数据格式转换: 仅支持 double 类型数据, COO 格式转换为 CSR 格式。

参数说明:

nnz:非零元个数;

n_rows:稀疏矩阵行数;

rows:稀疏矩阵行索引号指针;

rows_off:稀疏矩阵行偏移指针;

(7) void SWAF_spmv_r2o_d(int *nnz, int n_rows, int *n_cols, T_idx *rows_off, T_idx *cols, T_idx **rows);

函数说明:

数据格式转换: 仅支持 double 类型数据, CSR 格式转换为 COO 格式。

参数说明：

nnz:非零元个数指针；

n_rows:稀疏矩阵行数；

n_cols:稀疏矩阵列数指针；

rows_off:稀疏矩阵行偏移指针；

cols:稀疏矩阵列索引号指针；

rows:稀疏矩阵行索引号指针。

四、程序示例

1、COO 格式矩阵向量乘法

下图框中所示源代码，其中 42 行代码为读文件，第 60 行代码和第 61 行代码功能为 COO 格式存储稀疏矩阵向量乘的众核运算，第 68 行代码为 COO 格式存储稀疏矩阵向量乘主核运算，第 95 行代码功能为销毁内存。

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <string.h>
4 #include <crts.h>
5
6 #include "spmv.h"
7
8 void sigfunc(int sig)
9 {
10     printf("kasile\n");
11     while(1);
12 }
13
14 unsigned long real_size;
15
16 int main(int argc, char *argv[])
17 {
```

```
18  signal(11, sigfunc);
19  CRTS_init();
20
21  char *file_name,*dtype;
22  char file_name_wr[MAX_BUFFER] = "../CheckFiles/checkread_";
23  FILE* fp;
24  char match[MAX_BUFFER] = "%./";
25
26  int type_size = -1,real_type_size=-1,point_length = -1;
27  int i,ierr ;
28
29  T_data *data;
30  T_idx *rows,*cols,*rows_off;
31  char *wr_name;
32  int n_rows,n_cols,nnz;
33
34  if(argc==0)
35  {
36      printf("please input filename !!!!! \n");
37      return 0 ;
38  }
39  else
40      file_name = argv[1];
41
42  ierr = read_MTX(&data,&rows,&cols,&nnz,&n_rows,&n_cols,file_name);
43  printf("nnz = %d, n_rows = %d,n_cols = %d\n",nnz,n_rows,n_cols);
44
45  double *y = (double*)malloc(n_rows*sizeof(double));
46  double *x = (double*)malloc(n_cols*sizeof(double));
47  for(i=0; i<n_rows; i++)
48      y[i] = 0.0;
49
50  for(i=0; i<n_cols; i++)
51      x[i] = 1.2344324 * i;
52
53  for(i=0;i<nnz;i++)
54  {
55      rows[i] = rows[i]-1;
56      cols[i] = cols[i]-1;
57  }
58  _spmv_data_t *_data = (spmv_data_t*)malloc(sizeof(spmv_data_t));
59
60  SWAF_spmv_init_coo_d(n_rows, n_cols, nnz, rows, cols, data, _data);
61  SWAF_spmv(_data, x, y);
```

```
62
63     double *y_res = (double*)malloc(n_rows * sizeof(double));
64
65     for(i=0; i<n_rows; i++)
66         y_res[i] = 0.;
67     int ii, jj;
68     SWAF_spmv_coo_master_d(nnz, rows, cols, data, x, y_res);
69
70     int correct = 1;
71     for(i=0; i<n_rows; i++)
72         if(y[i] != y_res[i])
73         {
74             correct = 0;
75             break;
76         }
77     if(correct)
78     {
79         printf("SPMV result correct!\n");
80     }
81     else
82     {
83         FILE *fp_res = fopen("y_slv", "w");
84         double res = 0.;
85         for(i=0; i<n_rows; i++)
86         {
44,2           70%
85         for(i=0; i<n_rows; i++)
86         {
87             res += y[i];
88             if(y[i] != y_res[i])
89                 fprintf(fp_res, "y[%d]=%.15lf, y_res[%d]=%.15lf\n", i, y[i], i, y_res[i]);
90         }
91         fclose(fp_res);
92         printf("sum of y is %lf \n", res);
93     }
94
95     SWAF_spmv_finish(_data);
96     free(data);
97     free(rows);
98     free(cols);
99     free(y);
100    free(x);
101    free(y_res);
```

```
102
103 }
```

上述代码编译时需要添加 `spmv.h` 头文件目录，链接时需要添加库路径。具体参见下述命令。

```
sw9gcc -mieee -O3 -Woverflow -I./include -c main.c -o main.o
```

```
sw9gcc -mhybrid main.o ./lib/libSWAF_spmv.a -o EX1.exe
```

2、CSR 格式矩阵向量乘法

下图框中所示源代码，其中 41 行代码为读文件，第 70 行代码和第 71 行代码功能为 COO 格式存储稀疏矩阵向量乘的众核运算，第 78 行代码为 COO 格式存储稀疏矩阵向量乘主核运算，第 104 行代码功能为销毁内存。

```
1 #include <stdio.h>
2 #include <math.h>
3 #include <string.h>
4 #include <crt.h>
5 #include "spmv.h"
6
7 void sigfunc(int sig)
8 {
9     printf("kasile\n");
10    while(1);
11 }
12
13 unsigned long real_size;
14
15 int main(int argc, char *argv[])
16 {
17     signal(11, sigfunc);
18     CRTS_init();
19
20     char *file_name,*dtype;
21     char file_name_wr[MAX_BUFFER] = "../CheckFiles/checkread_";
22     FILE* fp;
23     char match[MAX_BUFFER] = "%./";
24
25     int type_size = -1,real_type_size=-1,point_length = -1;
26     int i,j,ierr ;
27
28     T_data *data;
```

```

29  T_idx *rows,*cols,*rows_off,*pre_cols;
30  char *wr_name;
31  int n_rows,n_cols,nnz;
32
33  if(argc==0)
34  {
35      printf("please input filename !!!!! \n");
36      return 0 ;
37  }
38  else
39      file_name = argv[1];
40
41                                     ierr =
read_MTX(&data,&rows,&pre_cols,&nnz,&n_rows,&n_cols,file_name);
42  printf("nnz = %d, n_rows = %d,n_cols = %d\n",nnz,n_rows,n_cols);
43
44  cols = (T_idx*)malloc(nnz*sizeof(T_idx));
45  double *y = (double*)malloc(n_rows*sizeof(double));
46  double *x = (double*)malloc(n_cols*sizeof(double));
47  for(i=0;i<nnz;i++)
48  {
49      cols[i] = rows[i];
50      rows[i] = pre_cols[i];
51  }
52
53  for(i=0; i<n_rows; i++)
54      y[i] = 0.0;
55
56  for(i=0; i<n_cols; i++){
57      x[i] = 1.2344324 * i;
58  }
59
60  for(i=0;i<nnz;i++)
61  {
62      rows[i] = rows[i]-1;
63      cols[i] = cols[i]-1;
64  }
65
66  _spmv_data_t *_data = (spmv_data_t*)malloc(sizeof(spmv_data_t));
67
68  SWAF_spmv_o2r_d(nnz,n_rows,rows,&rows_off);
69
70  SWAF_spmv_init_csr_d(n_rows, data, cols, rows_off, _data);
71  SWAF_spmv(_data, x, y);

```

```

72
73  double *y_res = (double*)malloc(n_rows * sizeof(double));
74  for(i=0; i<n_rows; i++)
75      y_res[i] = 0.;
76  int ii, jj;
77
78  SWAF_spmv_csr_master_d(n_rows, rows_off, cols, data, x, y_res);
79  int correct = 1;
80  for(i=0; i<n_rows; i++)
81      if(y[i] != y_res[i])
82      {
83          correct = 0;
84          break;
85      }
86  if(correct)
87  {
88      printf("SPMV result correct!\n");
89  }
90  else
91  {
92      FILE *fp_res = fopen("y_slv", "w");
93      double res = 0.;
94      for(i=0; i<n_rows; i++)
95      {
96          res += y[i];
97          if(y[i] != y_res[i])
98              fprintf(fp_res, "y[%d]=%.15lf, y_res[%d]=%.15lf\n", i, y[i], i, y_res[i]);
99      }
100     fclose(fp_res);
101     printf("sum of y is %lf \n", res);
102 }
103
104 SWAF_spmv_finish(_data);
105 free(data);
106 free(rows);
107 free(cols);
108 free(pre_cols);
109 free(y);
110 free(x);
111 free(y_res);
112
113 }

```

上述代码编译时需要添加 `spmv.h` 头文件目录，链接时需要添加库路径。具体参见下述命

令。

```
sw9gcc -mieee -O3 -Woverflow -I./include -c main.c -o main.o
```

```
sw9gcc -mhybrid main.o ./lib/libSWAF_spmv.a -o EX2.exe
```

五、性能测试

性能测试数据选取了 8 组 MTX 数据，测试结果如下表 1 所示。

表 1 矩阵向量乘性能测试

Number	Mat_name	Rows	coo	csr
1	2cubes_sphere	101492	9.324959	7.038262
2	G2_circuit	150102	11.352152	21.900777
3	G3_circuit	1585478	6.585538	10.712411
4	apache2	715176	8.168080	13.798230
5	atmosmodd	1270432	8.637036	8.402478
6	ecology2	999999	7.473931	6.748858
7	Zhao1/Zhao1.mtx	33861	3.428217	3.519750
8	marine1/marine1.mtx	400320	7.966623	6.748858

第四章、稠密矩阵乘使用手册

一、简介

本手册提供双精度实数矩阵乘、单精度复数矩阵乘以及半精度复数矩阵乘接口，其他情况请用神威高性能数学库。双精度实数矩阵乘对于矩阵按行存储与按列存储两种格式分别提供调用接口；单精度复数矩阵乘接口（行优先存储），对于矩阵 A 和矩阵 B 需要转置的情况分别提供单独的接口；半精度复数矩阵乘接口，暂时只提供矩阵 B 需要转置的情况。由于主核上不支持半精度，半精度矩阵乘提供 2 种接口，一种是主核上矩阵 A 和矩阵 B 存储数据为单精度数据，结果矩阵 C 为单精度数据；另一种存储数据为半精度数据，结果矩阵 C 为半精度数据。

二、使用方法

使用时需要包含/usr/lib/SWAF 路径下 include 目录下的 SWAF_gemm.h,并链接 lib 目录下的 libSWAF_gemm.a。

三、编程接口

(1) void SWAF_cgemm(const int M, const int N, const int K, const void *A, const void *B, void *C);

函数说明：

单精度复数矩阵乘 $A*B$ 。

参数说明：

M: 矩阵 A 和矩阵 C 的行数；

N: 矩阵 B 和矩阵 C 的列数；

K: 矩阵 A 的列数和矩阵 B 的行数；

A: 输入矩阵 A；

B: 输入矩阵 B;

C: 结果矩阵 C;

```
(2) void SWAF_cgemm_TransB(const int M, const int N, const int K, const void *A,  
const void *B, void *C);
```

函数说明:

单精度复数矩阵乘 $A*B$ (转置)。

参数说明:

M: 矩阵 A 和矩阵 C 的行数;

N: 矩阵 B 的行数和矩阵 C 的列数;

K: 矩阵 A 的列数和矩阵 B 的列数;

A: 输入矩阵 A;

B: 输入矩阵 B;

C: 结果矩阵 C;

```
(3) void SWAF_cgemm_TransA(const int M, const int N, const int K, const void *A,  
const void *B, void *C);
```

函数说明:

单精度复数矩阵乘 A (转置) $*B$ 。

参数说明:

M: 矩阵 A 的列数和矩阵 C 的行数;

N: 矩阵 B 的列数和矩阵 C 的列数;

K: 矩阵 A 的行数和矩阵 B 的行数;

A: 输入矩阵 A;

B: 输入矩阵 B;

C: 结果矩阵 C;

(4) void SWAF_cgemm_TransAB(const int M, const int N, const int K, const void *A, const void *B, void *C);

函数说明:

单精度复数矩阵乘 A (转置) * B (转置)。

参数说明:

M: 矩阵 A 的列数和矩阵 C 的行数;

N: 矩阵 B 的行数和矩阵 C 的列数;

K: 矩阵 A 的行数和矩阵 B 的列数;

A: 输入矩阵 A;

B: 输入矩阵 B;

C: 结果矩阵 C;

(5) void SWAF_cgemm_trans_hf(const int M, const int N, const int K, const void *A, const void *B, void *C);

函数说明:

半精度复数矩阵乘 $A*B$ ，输入矩阵 A 、输入矩阵 B 以及输出矩阵 C 均为单精度数据，从核计算用半精度计算。

参数说明：

M: 矩阵 A 和矩阵 C 的行数；

N: 矩阵 B 和矩阵 C 的列数；

K: 矩阵 A 的列数和矩阵 B 的行数；

A: 输入矩阵 A ；

B: 输入矩阵 B ；

C: 结果矩阵 C ；

(6) `void SWAF_cgemm_TransB_trans_hf(const int M, const int N, const int K, const void *A, const void *B, void *C);`

函数说明：

半精度复数矩阵乘 $A*B$ （转置），输入矩阵 A 、输入矩阵 B 以及输出矩阵 C 均为单精度数据，从核计算用半精度计算。

参数说明：

M: 矩阵 A 和矩阵 C 的行数；

N: 矩阵 B 的行数和矩阵 C 的列数；

K: 矩阵 A 的列数和矩阵 B 的列数；

A: 输入矩阵 A ；

B: 输入矩阵 B ；

C: 结果矩阵 C;

(7) void SWAF_cgemm_hf(const int M, const int N, const int K, const void *A, const void *B, void *C);

函数说明:

半精度复数矩阵乘 $A*B$, 输入矩阵 A、输入矩阵 B 以及输出矩阵 C 均为半精度数据。

参数说明:

M: 矩阵 A 和矩阵 C 的行数;

N: 矩阵 B 和矩阵 C 的列数;

K: 矩阵 A 的列数和矩阵 B 的行数;

A: 输入矩阵 A;

B: 输入矩阵 B;

C: 结果矩阵 C;

(8) void SWAF_cgemm_TransB_hf(const int M, const int N, const int K, const void *A, const void *B, void *C);

函数说明:

半精度复数矩阵乘 $A*B$ (转置), 输入矩阵 A、输入矩阵 B 以及输出矩阵 C 均为半精度数据。

参数说明:

M: 矩阵 A 和矩阵 C 的行数;

N: 矩阵 B 的行数和矩阵 C 的列数;

K: 矩阵 A 的列数和矩阵 B 的列数;

A: 输入矩阵 A;

B: 输入矩阵 B;

C: 结果矩阵 C;

(9) void SWAF_RowMajor_dgemm(const SWAF_TRANSPOSE TransA, const SWAF_TRANSPOSE TransB, const int M, const int N, const int K, const double alpha, const void *A, const int lda, const void *B, const int ldb, const double beta, void *C, const int ldc);

函数说明:

$$C = \alpha * A * B + \beta * C$$

参数说明:

TransA: 当 TransA= SwafTrans, 则矩阵 A 需要转置; 当 TransA= SwafNoTrans, 则矩阵 A 不需要转置;

TransB: 当 TransB= SwafTrans, 则矩阵 B 需要转置; 当 TransB= SwafNoTrans, 则矩阵 B 不需要转置;

M: 矩阵乘法中 A 和矩阵 C 的行数

N: 矩阵乘法中 B 和矩阵 C 的列数

K: 矩阵乘法中 A 的列数和矩阵 B 的行数

alpha: 矩阵 A*B 的系数

A: 输入矩阵 A

lda: 当 TransA= SwafNoTrans 时, lda=K; 当 TransA= SwafTrans 时, lda=M

B: 输入矩阵 B

ldb: 当 TransB= SwafNoTrans 时, lda=N; 当 TransB= SwafTrans 时, lda=K

beta: 矩阵 C 的系数

C: 结果矩阵 C

ldc: ldc=N

```
(10) void SWAF_ColMajor_dgemm(const SWAF_TRANSPOSE TransA, const SWAF_TRANSPOSE
TransB, const int M, const int N, const int K, const double alpha, const void *A,
const int lda, const void *B, const int ldb, const double beta, void *C, const int
ldc);
```

函数说明:

$$C = \alpha * A * B + \beta * C$$

参数说明:

TransA: 当 TransA= SwafTrans, 则矩阵 A 需要转置; 当 TransA= SwafNoTrans, 则矩阵 A 不需要转置;

TransB: 当 TransB= SwafTrans, 则矩阵 B 需要转置; 当 TransB= SwafNoTrans, 则矩阵 B 不需要转置;

M: 矩阵乘法中 A 和矩阵 C 的行数

N: 矩阵乘法中 B 和矩阵 C 的列数

K: 矩阵乘法中 A 的列数和矩阵 B 的行数

alpha: 矩阵 A*B 的系数

A: 输入矩阵 A

lda: 当 TransA= SwafNoTrans 时,lda=M; 当 TransA= SwafTrans 时,lda=K

B: 输入矩阵 B

ldb: 当 TransB= SwafNoTrans 时,lda=K; 当 TransB= SwafTrans 时,lda=N

beta:矩阵 C 的系数

C: 结果矩阵 C

ldc: ldc=M

四、性能数据

1、单精度

单精度复数矩阵乘整体性能如表 1 所示。

表 1 单精度矩阵乘整体性能

A*B				
M	N	K	拍数	Tflops
16 ⁴	16 ²	16	29173343	0.15
16 ⁴	16	16 ²	25972484	0.17
16 ⁴	16 ²	16 ²	51924920	1.39
16 ³	16 ²	16 ²	3421737	1.32
16 ³	16 ³	16 ³	549721493	2.1

2、半精度

半精度复数矩阵乘整体性能如表 2 所示。

表 2 半精度矩阵乘整体性能

			A*B			
M	N	K	拍数（半精度数据）	Tflops	拍数（单精度数据）	Tflops
16 ⁴	16 ²	16	39373065	0.11	66567974	0.07
16 ⁴	16	16 ²	18656793	0.24	33613577	0.13
16 ⁴	16 ²	16 ²	31895830	2.26	71877168	1
16 ³	16 ²	16 ²	2179764	2.07	4664583	0.97

16^3	16^3	16^3	167013547	6.91	219514667	5.26
------	------	------	-----------	------	-----------	------

3、双精度

双精度实数矩阵乘整体性能如表 3 所示。

表 3 半精度矩阵乘整体性能

C=A*B				
M	N	K	拍数	Tflops
64	64	64	120542	0.01
128	128	128	130659	0.07
256	256	256	209127	0.33
512	512	512	555455	1.01
1024	1024	1024	2818481	1.6
2048	2048	2048	19928874	1.81
4096	4096	4096	149048028	1.93

第五章、张量转置

一、简介

本手册提供单精度张量转置功能，分为 6 阶、5 阶、4 阶张量转置 3 个接口，其中 6 阶张量转置需要使用大共享模式。

以 6 阶张量为例，初始轴的位置是 (0,1,2,3,4,5)，使用接口，设置参数 index 为 041234，转置后的张量轴的位置是 (0,4,1,2,3,5)。

$C(i_0, i_1, i_2, i_3, i_4, i_5)$  $C(i_0, i_4, i_1, i_2, i_3, i_5)$

6 阶张量转置已实现的情况包括：

014235，041235，051234，123450，501234

5 阶张量转置已实现的情况包括：

01324，02314，03124，23014

4 阶张量转置已实现的情况包括：

0213，1023，2013

其他情况，如果有需要请联系技术支持（见附录 2）。

二、使用方法

使用时需要包含/usr/lib/SWAF 路径下 include 目录下的 SWAF_transpose.h,并链接 lib 目录下的 libSWAF_transpose.a。

三、编程接口

(1) void SWAF_transpose_4(const void * Input,void * Output,const char * Index);

函数说明：

4 阶张量转置。

参数说明：

Input: 输入张量;

Output: 转置后的输出张量;

Index: 转置后张量的轴的位置;

(2) void SWAF_transpose_5(const void * Input,void * Output,const char * Index);

函数说明:

5 阶张量转置。

参数说明:

Input: 输入张量;

Output: 转置后的输出张量;

Index: 转置后张量的轴的位置;

(3) void SWAF_transpose_6(const void * Input,void * Output,const char * Index);

函数说明:

6 阶张量转置。

参数说明:

Input: 输入张量;

Output: 转置后的输出张量;

Index: 转置后张量的轴的位置;

四、程序示例

以 5 阶张量转置其中一种情况为例, 程序示例如下:

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <crts.h>
4 #include <complex.h>
5 #include <time.h>
6 #include <assert.h>
7 #include <SWAF_transpose.h>
8
9 int main(int argc, char* argv[])
10 {
11     int Size=33554432;//32^5
12     complex float* input = (complex float*)malloc(Size*sizeof(complex float));
13     assert(input!=NULL);
14     complex float* output_master = (complex float*)malloc(Size*sizeof(complex float));
15     assert(output_master!=NULL);
16     complex float* output_slave = (complex float*)malloc(Size*sizeof(complex float));
17     assert(output_slave!=NULL);
18
19     complex float tmp;
20     srand((unsigned)time(NULL));
21     int i,j,k;
22     for(i=0; i<Size; i++)
23         input[i] = rand()%20+rand()%20*_Complex_I;
24
25     CRTS_init();
26     unsigned long st, ed;
27     asm volatile("memb\n");
28     printf("*****\n");
29     printf("SWAF_transpose begin-----\n");
30     st = CRTS_time_cycle();
31     SWAF_transpose_5(input,output_slave,"02314");
32     ed = CRTS_time_cycle();
33     unsigned long time_slv;
34     time_slv = ed - st;
35     printf("time in SWAF_transpose :\t\t%d\n", time_slv);
36
37     printf("*****\n");
38     printf("master begin-----\n");
```

```
39  int right = 1;
40  int L=32;
41  int i1,i2,i3,i4,i5,w1,w2;
42  st = CRTS_time_cycle();
43  for(i1=0;i1<L;i1++)
44  for(i2=0;i2<L;i2++)
45  for(i3=0;i3<L;i3++)
46  for(i4=0;i4<L;i4++)
47  for(i5=0;i5<L;i5++)
48  {
49      w1=i1*L*L*L*L+i3*L*L*L+i4*L*L+i2*L+i5;
50      w2=i1*L*L*L*L+i2*L*L*L+i3*L*L+i4*L+i5;
51      output_master[w1]=input[w2];
52  }
53
54  ed = CRTS_time_cycle();
55  unsigned long time_master;
56  time_master = ed - st;
57  FILE* fp;
58  if((fp = fopen("error","w")) == NULL)
59  {
60      printf("Error open file error!\n");
61      exit(0);
62  }
63  for(i=0; i<Size; i++)
64  {
65      if(output_master[i] != output_slave[i])
66      {
67
68          fprintf(fp,"output_slave[%d]=%f,%f\n",i,creal(output_slave[i]),cimag(output_slave[i]));
69
70          fprintf(fp,"output_master[%d]=%f,%f\n",i,creal(output_master[i]),cimag(output_master[i]));
71
72          right = 0;
73          break;
74      }
75  }
```

```
75     printf("time in master :\t\t%d\n", time_master);
76     printf("*****\n");
77     printf("result is right!\n");
78     printf("speedup :\t\t%.2lf\n", 1.0 * time_master / time_slv);
79     printf("*****\n");
80 }
81 free(input);
82 free(output_master);
83 free(output_slave);
84 return 0;
85 }
```

五、性能对比

单核组加速比如下表所示：

	转置后轴的位置	主核拍数	众核拍数	加速比	带宽（GB/s）
5 阶张量	（2,3,0,1,4）	1133462160	23385038	48.47	47.21

第六章、窄条矩阵乘

一、使用方法

使用时需要包含/usr/lib/SWAF 路径下 include 目录下的 SWAF_ngemm.h,并链接 lib 目录下的 libSWAF_ngemm.a。

二、函数接口

void SWAF_ngemm(int M, int N, int K, double *A, double *B, double *C);

函数说明：

窄条矩阵乘函数。

参数说明：

M: 矩阵 A 与 C 的行数；

N: 矩阵 B 与 C 的列数；

K: 矩阵 A 的列数/矩阵 B 的行数；

A: 输入矩阵 A；

B: 输入矩阵 B；

C: 输出矩阵 C；

返回值：

成功：0；

失败：非 0。

三、程序示例

以 A[2048×2]×B[2×2048]的窄条矩阵乘为例，众核程序示例如下：

```
1      #include <stdlib.h>
2      #include <stdio.h>
3      #include <crt.h>
4      #include "dgemm.h"
5      #include <stdlib.h>
6
7
8      int main(int argc, char** argv){
9          int M, N, K;
10         if(argc != 4)
11             {
12                 printf("please input M, N, K!\n");
```

```
13         exit(0);
14     }
15     M = atoi(argv[1]);
16     N = atoi(argv[2]);
17     K = atoi(argv[3]);
18     printf("M=%d, N=%d, K=%d\n", M, N, K);
19     double* A = (double*)malloc(M*K*sizeof(double));
20     double C[M*N];
21     double B[K*N];
22     int i;
23     for(i=0; i<K; i++)
24         A[i] = 1.0;
25     for(i=K; i<2*K; i++)
26         A[i] = 2.0;
27     for(i=2*K; i<M*K; i++)
28         A[i] = 3.0;
29     for(i=0; i<N; i++)
30         B[i] = 1.0;
31     for(i=N; i<K*N; i++)
32         B[i] = 2.0;
33     for(i=0; i<M*N; i++)
34         C[i] = 0.0;
35     CRTS_init();
36     unsigned long st, ed;
37     st = CRTS_time_cycle();
38     SWAF_ngemm(M, N, K, A, B, C);
39     ed = CRTS_time_cycle();
40     time_slv = ed - st;
41     #ifdef check_res
42     int j, k;
43     int right = 1;
44     double* C_new = (double*)malloc(M*N*sizeof(double));
45     for(i=0; i<M*N; i++)
46         C_new[i] = 0.0;
47     st = CRTS_time_cycle();
48     for(i=0; i<M; i++)
49         for(j=0; j<N; j++)
50             for(k=0; k<K; k++)
51                 C_new[i*N+j] += A[i*K+k] * B[k*N+j];
52     ed = CRTS_time_cycle();
53     unsigned long time_master;
54     time_master = ed - st;
55 }
```

四、性能对比

相比于主核，众核版窄条矩阵乘的但合租加速比如下表所示：

	主核版	众核版	加速比
拍数	235433157	3832125	61.4 倍

第七章、三维矩阵转置

一、使用方法

使用时需要包含/usr/lib/SWAF 路径下 include 目录下的 SWAF_swap.h,并链接 lib 目录下的 libSWAF_swap.a。

二、函数接口

void SWAF_swap (__DATA_TYP* input, __DATA_TYP* output, int L, int M, int N)

函数说明：

矩阵转置函数。

参数说明：

input: 输入矩阵；

output: 转置后的输出矩阵；

L: 矩阵的长；

M: 矩阵的宽；

N: 矩阵的高；

返回值：

成功：0；

失败：非 0。

三、程序示例

以[128×512×512]的三维矩阵转置为例，其众核程序示例如下：

```
1      #include <stdlib.h>
2      #include <stdio.h>
3      #include <math.h>
4      #include <crts.h>
5      int main(int argc, char* argv[]){
6
7          double ***a,***bs,***bm;
8          int i,j,k;
9          unsigned long t1,t2,t3,t4,t0;
10
11          int L = atoi(argv[1]);
12          int M = atoi(argv[2]);
13          int N = atoi(argv[3]);
```

```

14
15         double tsize=(double)L*(double)M*(double)N;
16         double tmp_size = tsize*sizeof(double);
17         tsize = (tsize*sizeof(double))/(1024.0*1024.0);
18
19         a  = array3DD(L,M,N);
20         bs = array3DD(L,N,M);
21         bm = array3DD(L,N,M);
22     CRTS_init();
23     t0 = CRTS_time_cycle();
24     for(i=0; i<L; i++){
25         for(j=0; j<M; j++){
26             for(k=0; k<N; k++){
27                 a[i][j][k] = 10.0*i + 1.0*j + 0.0001*k;
28                 bs[i][k][j] = 0;
29             }
30         }
31     }
32     t1 = CRTS_time_cycle();
33
34     for(i=0; i<L; i++){
35         for(j=0; j<M; j++){
36             for(k=0; k<N; k++){
37                 bm[i][k][j] = a[i][j][k];
38             }
39         }
40     }
41
42     SWAF_swap_init();
43     t2 = CRTS_time_cycle();
44     SWAF_swap(&(a[0][0][0]), &(bs[0][0][0]), L, M, N);
45     t3 = CRTS_time_cycle();
46     unsigned long flag_err = 0;
47     unsigned long flag = 0;
48     printf("          L *          M *          N |      Malloc-Size |
M-Cycle-Time          S-Cycle-Time          Speed-Up
\n",L,M,N,(tsize*3),t2-t1,t3-t2,1.0*(t2-t1)/(t3-t2));
49     for(i=0; i<L; i++){
50         for(k=0; k<M; k++){
51             for(j=0; j<N; j++){
52                 flag = flag + 1;
53                 if(abs(bs[i][k][j] - bm[i][k][j])>0){
54                     flag_err = flag_err+1;
55                 }

```

```
56         }
57     }
58 }
59 SWAF_swap_finalize();
60 arrayFree3DD(a);
61     arrayFree3DD(bs);
62 athread_halt();
63     return 0;
64 }
```

四、性能对比

相比于主核，众核版三维矩阵转置的单核组加速比如下表所示：

	主核版	众核版	加速比
拍数	2484651830	57190427	43.45 倍

第八章、稀疏对角矩阵方程组求解

一、使用方法

使用时需要包含/usr/lib/SWAF 路径下 include 目录下的 SWAF_pipmat.h,并链接 lib 目录下的 libSWAF_pipmat.a。

二、函数接口

void SWAF_forward_2D5(double *X, double *A, double *B, int M, int N, int L)

函数说明:

2 维矩阵, 5 对角线的稀疏对角矩阵方程组求解。

参数说明:

X: 解向量;
A: 稀疏矩阵;
B: 方程右端系数;
M: 矩阵的长;
N: 矩阵的宽;
L: 矩阵的高;

返回值:

成功: 0;

失败: 非 0。

void SWAF_forward_3D7(double *X, double *A, double *B, int M, int N, int L)

函数说明:

3 维矩阵, 7 对角线的稀疏对角矩阵方程组求解。

参数说明:

X: 解向量;
A: 稀疏矩阵;
B: 方程右端系数;
M: 矩阵的长;
N: 矩阵的宽;
L: 矩阵的高;

返回值:

成功: 0;

失败: 非 0。

三、程序示例

以[8000×6000]的二维矩阵 5 对角线的稀疏对角矩阵方程组求解为例，其众核程序示例如下

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include "func.h"
4      #include <crt.h>
5      int main(int argc, char* argv[]){
6
7          signal(8,sigfunc);
8          //int STEPS = 5;
9          int STEPS = 1;
10         char *type = argv[1];
11         double SIZE;
12         const double FREQ = 1.9E9;
13         void (*base)      (double *, double *, double *, int, int,
14         int) = NULL;
15         void (*forward)   (double *, double *, double *, int, int,
16         int) = NULL;
17         void (*SWAF_ILU)   (double *, double *, double *, int, int,
18         int) = NULL;
19
20         double *A = NULL;
21         double *B = NULL;
22         double *X1 = NULL;
23         double *X2 = NULL;
24
25         int rnz = 0;
26         int s;
27         unsigned long tt[2][100];
28         unsigned long t_mean[2];
29         int i,h;
30         t_mean[0] = 0;
31         t_mean[1] = 0;
32         int M=1, N=1, L=1;
33         if(argc > 2) M = atoi(argv[2]);
34         if(argc > 3) N = atoi(argv[3]);
35         if(argc > 4) L = atoi(argv[4]);
36
37         SIZE = 1.0*M*N*L*sizeof(double)*0.001*0.001*0.001;
38
39         pmat_init();
```

```

37         if(strcmp(type, "2D5") == 0)
38         {
39             rnz          = 2;
40             base         = SWAF_forward_2D5_MPE;
41             forward      = SWAF_forward_2D5;
42             //printf("Type:%s, <M,N>: <%d,%d> %.3lfMB\n", type,
M, N, (3+rnz)*SIZE);
43         }
44         if(strcmp(type, "3D7") == 0)
45         {
46             rnz          = 3;
47             base         = SWAF_forward_3D7_MPE;
48             forward      = SWAF_forward_3D7;
49             printf("Type:%s, <M,N,L>: <%d,%d,%d> %.3lfMB\n",
type, M, N, L, (3+rnz)*SIZE);
50         }
51
52         A  = (double*)malloc(rnz*M*N*L*sizeof(double));
53         B  = (double*)malloc(M*N*L*sizeof(double));
54         X1 = (double*)malloc(M*N*L*sizeof(double));
55         X2 = (double*)malloc(M*N*L*sizeof(double));
56
57         init(A, M*N*L*rnz);
58         init(B, M*N*L);
59         printf("                                \n");
60         printf(" begin without SWAF_ILU          \n");
61         printf("                                \n");
62         for(s = 0; s < STEPS; s++)
63         {
64             unsigned long t1, t2;
65             init0(X2, M*N*L);
66             t1=CRTS_time_cycle();
67             base(X2, A, B, M, N, L);
68             t2=CRTS_time_cycle();
69             tt[0][s] = t2-t1;
70             t_mean[0] += tt[0][s];
71         }
72         t_mean[0] = t_mean[0]/STEPS;
73         SWAF_ILU = SWAF_forward_2D5;
74         for(s = 0; s < STEPS; s++)
75         {
76             unsigned long t1, t2;
77             init0(X1, M*N*L);
78             t1=CRTS_time_cycle();

```

```
79          SWAF_ILU(X1, A, B, M, N, L);
80          t2=CRTS_time_cycle();
81          tt[1][s] = t2-t1;
82          t_mean[1] += tt[1][s];
83      }
84      t_mean[1] = t_mean[1]/STEPS;
85      printf(" SWAF_ILU Finalize          \n");
86      printf("
87          fflush(NULL);
88      double speedup = (1.0*t_mean[0])/t_mean[1];
89      check(X1, X2, M*N*L, M, N, L);
90      free(A);
91      free(B);
92      return 0;
93      }
```

四、性能对比

相比于主核，稀疏对角矩阵方程组求解的单核组加速比如下：

	主核版	众核版	加速比
拍数	2438992900	83927771	29.06 倍

附录 1：离散访存库应用实例

离散访存库在计算流体力学软件 `Code_Saturne`、中国空气动力研究发展中心高超声速软件 `AHL3D-uns` 以及西北某研究所的结构力学软件中得到应用。现以流体力学软件 `Code_saturne` 为例，介绍离散访存库使用方式。

一、非规则访存问题

非结构网格技术在航空航天、船舶工程、电磁模拟等众多科学与工程计算领域广泛应用。如何高效地解决非结构网格的离散访存问题一直是科学与工程计算并行算法和应用领域关注的核心热点问题之一。不同于计算密集和访存规律的稠密问题，以非结构网格为代表的稀疏类问题具有较低的计算访存比和不规律随机离散访存等特点。随着网格规模的增加，离散访存问题越来越突出，对计算效率产生的负面影响越来越大，尤其是国产异构众核架构，其访存瓶颈问题更加突出。

二、接口使用

为提高好用性，离散访存库提供了简洁的编程接口，如图 1 所示。

```
1.Extern void DIS_prePosmod(int *iter_table,\n                           int *pos_table, \n                           int iter_size, \n                           int data_span, \n                           struct DIS_TYPEDATA *pos2global);\n\n2.Extern void DIS_preprocess_join();\n\n3.Extern Void DIS_read(void *master, \n                      int slave_block_i, \n                      int vec_size, \n                      struct DIS_TYPEDATA *pos2global,\n                      void *slave_return );\n\n4.Extern void DIS_write(void *updata_table, \n                      void *slave_block_i,\n                      void *vec_size, \n                      struct DIS_TYPEDATA *pos2global,\n                      void *updata_master);
```

图 1 离散访存优化算法程序接口

图 2 为其中一个热点循环体数据离散访存示意。`cell_cen` 是对邻接顶点求差分，而邻接点的编码不连续，从程序中可以看出 `cell_cen` 右边顶点的编码是通过查 `cell_cells_lst` 表获取坐标值。

```

1. for cell 0 to n_cells do //遍历顶点

//遍历邻接顶点
2. for cidx cell_cells_idx[cell] to cell_cells_idx[cell+1] do

3. jj = cell_cells_1st[cidx]; //求邻接顶点编号

// 具体计算过程
4. for ll 0 to 3 do
5. tmp[ii]=0.;
6. dist[ll]= cell_cen[ii][ll] - cell_cen[jj][ll];
7 }
8. ...
9. Grist[jj] = max(dis[0], tmp[0]);

// 计算结束

```

图 2 热点函数中的离散访存问题

通过分析程序代码，使用离散访存库设计众核并行方案，如图 3 所示。图 3 左边是坐标预处理过程，在计算前调用 DIS_prePosmod 接口创建坐标映射模型。右边是对循环体众核并行化改造。DIS_read 与 DIS_write 接口通过消息队列模型基于寄存器通信轮转方式实现远程离散数据批量读写功能，是集片上通信与数据重排为一体的功能化接口，并与计算过程相互独立，因此在实际编程中无法利用双缓冲机制实现计算与通信重叠，但高效的异步并发算法可充分发挥片上带宽性能，减少通信开销。

创建坐标映射模型	离散读写接口使用
// 主核预处理 1. DIS_prePosmod 2. ... // 主核可以继续执行与坐标模型无关的操作 //等待预处理完成 3.DIS_preprocess_join;	1.for I 0 to cell_block do //对cell分块 2.Dis_read //调用离散访存库接口，获取数据 3.for j idx[i] to idx[i+1] do //遍历邻接顶点 //具体计算过程 4.for j idx[i] to idx[i+1] do 5. tmp[ii]=0.; 6. dist[ll]= cell_cen[ii][ll] - sort[pos_i][ll] 7. pos_i++; 8. endfor 9 ... 10.DIS_write //调用离散访存接口，写入数据 //计算结束

图 3 众核优化代码

三、性能模型与分析

假设本地从核与其余从核 RBA 通信的次数为 k^i （ i 表示核号）。设从核主频为 FR_{spe} GHz，从核访 LDM 带宽为 BW_{ldm} GB/s,平均筛出一个正确数据需要 C_{ldm} 次 ldm 访问，设置 2 个表示离散程度的变量 L_m^i 和 β_m^i ： L_m^i 表示第 i 号从核从其余从核进行第 m 次 RBA 的通信量(单位:byte)，那么 $\sum L_m^i$ 就表示第 i 号从核的消息总长， β_m^i 表示第 i 从核从其余从核进行第 m 次 RBA 的数据实际利用率， $\sum(L_m^i \times \beta_m^i)$ 表示第 i 号从核实际有效数据量。假设 RBA 的延迟为 T_{delay} 拍（拍数即为 CPU 时钟周期数）， k^i 表示本地从核与其余从核的通信总次数，则从核调用一次离散访存库的时间 T_{dcal} （单位：拍）满足：

$$\begin{aligned}
T_{dcal} &= t_0 + t_1 + \max_i \{t_2^i\} \\
&= T_0 + \frac{FR_{spe} M}{BW_{dma}} \\
&\quad + \max_i \left\{ \sum_{m=1}^{k^i} \left(\frac{FR_{spe} L_m^i}{BW_{rba}} + (FR_{spe} \cdot C_{ldm} \times L_m^i \times \beta_m^i) / BW_{ldm} \right) + k^i \cdot T_{delay} \right\}
\end{aligned}$$

四、性能测试

在 Code_Saturne 中，采用非结构网格进行计算，离散访存为主的计算占整体运行时间的 60% 以上。表 1 是不同网格规模（单核组规模）下优化后循环体总耗时分布（单位：秒）。

表 1 原始主核程序与调用离散访存库总耗时分布

网格规模(万)	原始主核程序	使用离散访存库	离散访存库预处理过程	单核组加速比(倍)
2	110.31	10.12	0.051	10.92
4	212.61	19.33	0.103	11.02
8	423.68	28.07	0.211	15.08
16	829.14	58.49	0.42	14.17

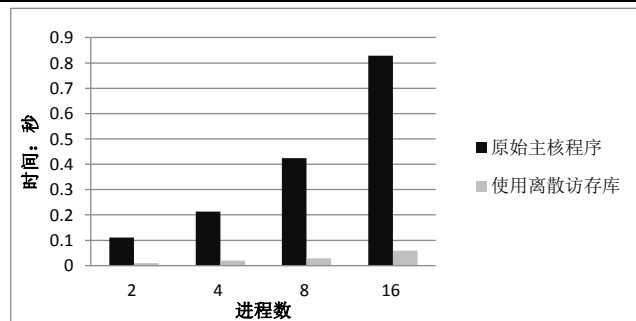


图 4 不同离散访存方法耗时分布图

图 4 直观地展示了表 1 的主要内容，其中，黑色柱形为原始程序，灰色柱形为调用离散访存库之后的优化程序。结合表 1 和图 4 可知在不同网格规模下，调用离散访存库的性能较原始程序比均有提升，8 万网格程序核心模块加速达到 15.08 倍。通过测试可知，预处理过程耗时可忽略不计，由于在以离散访存为主的计算热点优化前后占整体运行时间比值由原来 60% 下降至 8%，并使课题整体性能提升 100% 以上。由于 Code_Saturne 的性能瓶颈除离散访存计算之外还包括进程间的负载不均，通信等，因此为进一步提升应用整体性能，仍需要对课题进一步优化。

附录 2：技术支持

序号	功能	技术支持
1	SWAF_DCL	叶跃进
2	SWAF_PCG	叶跃进
3	SWAF_SPMV	郭恒、叶跃进
4	SWAF_GEMM	王臻
5	SWAF_TRANSPOSE	王臻
6	SWAF_NGEMM	郭恒
7	SWAF_SWAP	陈鑫
8	SWAF_PIPMAT	陈鑫